

TRABAJO PRÁCTICO INTEGRADOR**GESTIÓN DE DATOS DE PAÍSES EN PYTHON: FILTROS, ORDENAMIENTOS Y ESTADÍSTICAS****Alumnos**

Acevedo Ramiro – M26 C1-10

Nuñez Dario – M26 C1-26

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN**UNIVERSIDAD TECNOLÓGICA NACIONAL.****PROGRAMACIÓN I****Coordinador**

Alberto Cortez

Docente Tutor

Tomas Ferro

Tomas Garcia

14 de junio de 2026

CONTENIDO

MARCO TEÓRICO	4
• Listas	4
• Diccionarios	4
• Funciones	4
• Condicionales y estructuras repetitivas.....	5
• Ordenamientos	5
• Estadísticas básicas	5
• Archivos CSV	5
DECISIONES TÉCNICAS Y ARQUITECTURA.....	6
• DECISIONES TÉCNICAS.....	7
• FLUJO DE OPERACIONES PRINCIPALES.....	7
DIFICULTADES Y CONCLUSIONES.....	8
• DIFICULTADES ENCONTRADAS	8
• CONCLUSIONES	10
BIBLIOGRAFÍA / WEBGRAFÍA	11
ENLACES.....	11
• Link repo: https://github.com/Ramiroacev/Trabajo-Practico-Integrador-Prog-I	

- Link video:..... 11

MARCO TEÓRICO

- **Listas**

Las listas en Python son estructuras de datos dinámicas que permiten almacenar múltiples elementos en un solo contenedor. En este proyecto se utilizan para mantener la colección de países leídos desde el archivo CSV. Cada elemento de la lista corresponde a un país representado como un diccionario. Gracias a las listas, se pueden recorrer los registros con bucles, aplicar filtros y ordenamientos de manera sencilla.

- **Diccionarios**

Los diccionarios son estructuras que almacenan pares clave–valor. En el sistema, cada país se representa como un diccionario con las claves: nombre, población, superficie y continente. Esto facilita el acceso directo a los datos de cada país y permite manipularlos de forma clara y organizada, evitando confusiones entre atributos.

- **Funciones**

La modularización del código se logra mediante funciones, cada una con una responsabilidad específica: agregar un país, actualizar datos, buscar, filtrar, ordenar o mostrar estadísticas. Esto mejora la legibilidad, el mantenimiento y la reutilización del código. Además, permite que el menú principal invoque las funciones según la opción elegida por el usuario.

- **Condicionales y estructuras repetitivas**

El programa utiliza condicionales (if, match-case) para tomar decisiones según la entrada del usuario. También emplea bucles (while) para mantener el menú persistente y permitir múltiples operaciones sin reiniciar el programa. Estas estructuras son esenciales para controlar el flujo de ejecución y garantizar que el sistema responda correctamente a las distintas opciones.

- **Ordenamientos**

El ordenamiento de países se implementa con el método sort y la función lambda como clave. Esto permite ordenar por nombre, población o superficie, en forma ascendente o descendente. El ordenamiento es fundamental para organizar la información y facilitar la consulta de datos.

- **Estadísticas básicas**

El sistema calcula indicadores como el país con mayor y menor población, el promedio de población y superficie, y la cantidad de países por continente. Estas estadísticas se obtienen mediante funciones de agregación (max, min, sum) y operaciones sobre listas. Su objetivo es transformar los datos en información útil para el análisis.

- **Archivos CSV**

El archivo CSV es la fuente de datos del sistema. Se utiliza el módulo csv de Python para leer y escribir registros de manera estructurada. La lectura se realiza con DictReader, que convierte cada fila en un diccionario, y la escritura con DictWriter, que asegura que los datos se guarden con el formato correcto. El uso de CSV permite

mantener la persistencia de la información y facilita la interoperabilidad con otros sistemas.

DECISIONES TÉCNICAS Y ARQUITECTURA

La arquitectura del sistema se organizó siguiendo una estructura modular, con el objetivo de facilitar el mantenimiento, la reutilización del código y la separación de responsabilidades.

```
main.py
modulos/
  __init__.py
  paises.py
  utilidades.py
datos/
  paises_dataset.csv
```

Ilustración árbol proyecto

1. **main.py:** archivo principal que contiene el menú persistente y coordina las llamadas a las funciones.
2. **modulos/paises.py:** funciones específicas para la gestión de países (agregar, actualizar, buscar, filtrar, ordenar, estadísticas).
3. **modulos/utilidades.py:** funciones auxiliares como validaciones y entrada segura de datos.
4. **datos/paises_dataset.csv:** archivo base con los registros de países.

5. `__init__.py`: marca la carpeta `modulos` como paquete, permitiendo imports más claros.

- **DECISIONES TÉCNICAS**

- **Modularización:** cada funcionalidad se implementó como una función independiente, siguiendo el principio de “una función = una responsabilidad”.
- **Uso de diccionarios:** cada país se representa como un diccionario, lo que facilita el acceso a atributos por clave.
- **Persistencia de datos:** se eligió CSV como formato por su simplicidad y compatibilidad con otros sistemas.
- **Validaciones:** se implementaron controles de entrada para evitar errores (ejemplo: población y superficie deben ser enteros positivos).
- **Ordenamientos y filtros:** se usaron funciones de Python (`sort`, `filter`, `lambda`) para simplificar la lógica y mantener el código legible.
- **Menú persistente:** se utilizó un bucle `while` con `match-case` para mantener la interacción continua con el usuario.

- **FLUJO DE OPERACIONES PRINCIPALES**

El sistema sigue este flujo:

1. Inicio del programa → se ejecuta `main.py`, esto muestra el menú principal en consola.

```
-----  
                        MENU DE OPCIONES  
-----  
1: Agregar un País  
2: Actualizar datos (Población y Superficie)  
3: Buscar un País por nombre  
4: Filtrar Países  
5: Ordenar Países  
6: Mostrar Estadísticas  
7: Salir  
-----
```

2. Selección de opción → el usuario ingresa un número del 1 al 7.
3. Ejecución de función → según la opción, se llama a la función correspondiente en el modulo `países.py`.
4. Procesamiento de datos → se realizan operaciones sobre la lista de países (agregar, actualizar, filtrar, ordenar, estadísticas).
5. Salida en consola → se muestran resultados claros y legibles.

```
Ingrese una de las opciones del menú: 3  
Ingrese el nombre a buscar: argentina  
Nombre: Argentina, Población: 45376763, Superficie: 2780400 km², Continente: América
```

6. Retorno al menú → el programa vuelve al menú hasta que el usuario elija salir.

DIFICULTADES Y CONCLUSIONES

- **DIFICULTADES ENCONTRADAS**

Durante el desarrollo del sistema de gestión de países surgieron algunos obstáculos técnicos y organizativos:

- **Lectura del archivo CSV:** al inicio se presentaron problemas con el formato del archivo y la correcta interpretación de los datos, debido a la presencia de tildes en los nombres de países. Se resolvió utilizando estandarizando el módulo csv utf-8 de Python y validando los tipos de datos (enteros para población y superficie).
- **Validaciones de entrada:** fue necesario implementar controles para evitar que el usuario ingrese valores vacíos o incorrectos. Esto se solucionó con funciones auxiliares que verifican rangos y tipos de datos.
- **Modularización del código:** separar las funciones en distintos archivos (países.py, utilidades.py) requirió ajustar los imports y la estructura del proyecto. Esto ayuda no tener el conocido código espagueti. La inclusión de `__init__.py` ayudó a organizar el paquete para una futura actualización.
- **Ordenamientos y filtros:** al principio hubo confusión con el uso de sort y lambda para ordenar por distintos criterios. Se resolvió con ejemplos prácticos y pruebas sobre la lista de países.
- **Trabajo en equipo:** coordinar las tareas entre los integrantes implicó definir responsabilidades claras (uno trabajando en la lógica de funciones y otro en la documentación y pruebas).

- **CONCLUSIONES**

El proyecto permitió aplicar de manera integrada los conceptos de **listas, diccionarios, funciones, condicionales, ordenamientos y estadísticas** en un caso práctico. Las principales conclusiones son:

- La **modularización** facilita la comprensión y mantenimiento del código, ya que cada función cumple una responsabilidad específica.
- El uso de **diccionarios y listas** es una herramienta poderosa para representar y manipular datos estructurados como los países.
- La implementación de **validaciones** es clave para garantizar la robustez del sistema y evitar errores en la ejecución.
- El trabajo con **archivos CSV** muestra la importancia de la persistencia de datos y la interoperabilidad con otros sistemas.
- El proyecto fortaleció la capacidad de **trabajo colaborativo**, el uso de **GitHub** para control de versiones y la documentación técnica como parte esencial de la entrega.

En conclusión, el desarrollo del sistema de gestión de países no solo cumplió con los objetivos planteados en la consigna, sino que también permitió afianzar buenas prácticas de programación y organización de proyectos, aportando una experiencia valiosa para futuros trabajos académicos y profesionales.

BIBLIOGRAFÍA / WEBGRAFÍA

- Python Software Foundation. *Documentación oficial de Python 3*. Disponible en:
<https://docs.python.org/3/>
- Python Software Foundation. *Módulo csv — Lectura y escritura de archivos CSV*.
Disponible en: <https://docs.python.org/3/library/csv.html>
(docs.python.org)
- Universidad Tecnológica Nacional. *Material de Programación 1*. (Apuntes y guías de la cátedra)

ENLACES

- **Link repo:** <https://github.com/Ramiroacev/Trabajo-Practico-Integrador-Prog-I>
- **Link video:** <https://youtu.be/gidyWXaOABs>